



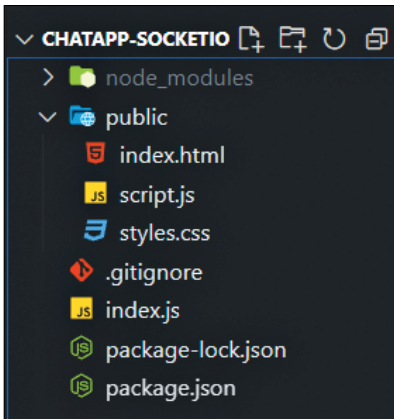
Period tracker on Whatsapp

Sirona, in association with Whatsapp, launched India's first period tracker. <https://dgit.in/jul22-63>



Xiaomi's first 2-In-1 Laptop Book

The Xiaomi Book S 12.4-inch is powered by the Qualcomm Snapdragon 8cx Gen 2 7nm chipset <https://dgit.in/jul22-64>



index.html file on the default '/' route. For those who are interested, the file structure that we followed is shown in the image above.

With the boilerplate out of the way, let's integrate socket.io to both the client side and the server side of our app.

INTEGRATING SOCKET.IO

Server Side

Adding socket.io to the server is really easy. Firstly, open up the terminal in your project directory. We're assuming the you've already ran the command CODE: `npm init -y` and installed all the dependencies such as express and nodemon. Now, to install socket.io, type in CODE: `npm install socket.io` and wait till the package gets downloaded and installed. Once that's done, open up your server file and you can require it there. CODE: `const socket_io = require("socket.io");`

Now, when you call CODE: `app.listen(PORT)` in your express app, it returns a server instance which you can then use to initialise the socket.io variable. Note that this method will only work if you have used Express to create a server. Hence, you can grab the server variable like this :-

```
const server = app;
listen(PORT, () => {
  console.log(`Listening on ${PORT}`)
})
```

And initialise your socket.io variable like this :- CODE: `const io = socket_io(server)` Finally, it should look something like this :-

CODE:

```
const express =
require("express")
const socket_io =
require("socket.io")
const PORT = 3000

const app = express()

const server = app;
listen(PORT, () => {
  console.log(`Listening on ${PORT}`)
})
```

```
const io = socket_io(server)
```

If you prefer creating servers the old way i.e. by using the http module, you can follow this code snippet from the socket.io docs. It's pretty similar to the stuff that we did above.

```
const express =
require("express");
const { createServer } =
require("http");
const { Server } =
require("socket.io");
```

```
const app = express();
const httpServer =
createServer(app);
const io = new
Server(httpServer, { /*
options */ });
```

```
httpServer.listen(3000);
```

You're pretty much good to go as far as server side integration is concerned.

Client Side

Since the web sockets created by socket.io will be communicating from client to the server and vice versa, we will also need to initialise the socket.io instance on the client side.

The first step is to get the latest version of the CODE: `socket.io.min.js` from socket.io's website. The link to that can be found here :- <https://dgit.in/socketio2>.

Just copy the whole code block shown on the website and paste it between the CODE: `<head> ... </head>` tags along with your other dependencies. Note that this method is only applicable if you're using HTML for the

frontend. If you're using a framework, it is recommended to install the CODE: `socket.io-client` package from npmjs. A link to that can be found here :- <https://dgit.in/socketio2>.

After doing this, simply go to your client side javascript file and write :-

```
const socket = io()
```

Note that the parenthesis in the `io()` usually contains a link to where the websocket has been initialised. Eg. `http://localhost:5500`. That's pretty much it. You have successfully initialised the socket.io instance on the client side as well. If you're using any framework, after doing CODE: `npm install socket.io-client` you can simply require / import it like this:-

```
import io from 'socket.io-client';
```

HELLO WORLD

Now that we've initialised socket.io on both the client side and the server side, it is time to write a simple hello world program to ensure that all the connections are successful.

These web sockets work almost the same as "event listeners" in javascript. When you attach an event listener to a particular element in the DOM, JS continuously listens for that event and triggers a function once that event has occurred. Hence, if your write

```
element.
addEventListener("click", (e)
=> {
  console.log("clicked")
})
```

Each time you click that element, "clicked" will get logged out to the console. Similarly, all the web sockets will listen to "events" which are emitted by other web sockets. An event can be either broadcasted or emitted and we'll see the difference between the two in a while. For now, in the client side, you can write this code after initialisation :-

```
socket.on("my-message",
(data) => {
  console.log(data)
})
```

And on the server side, you can input this code :-